

Name bhavya rana
Sapid 590034689
Batch 15

Algorithm

Main Program

1. Start.
2. Display the menu:
 - Verify associative property of matrix addition.
 - Verify associative property of matrix multiplication.
 - Exit.
3. Read the user's choice.
4. Use **switch**:
 - Choice 1 $\rightarrow$ call `additionAssociative()`.
 - Choice 2 $\rightarrow$ call `multiplicationAssociative()`.
 - Choice 3 $\rightarrow$ exit.
 - Otherwise $\rightarrow$ display an invalid-choice message.
5. Repeat until Exit is selected.
6. Stop.

Matrix Addition

1. Read positive values of m and n .
2. Declare three VLAs $A[m][n]$, $B[m][n]$, and $C[m][n]$.
3. Read all three matrices.
4. Calculate $A + B$, then calculate $(A+B) + C$.
5. Calculate $B + C$, then calculate $A + (B+C)$.
6. Compare corresponding elements of both result matrices.
7. Display both matrices and state whether the associative property is verified.

Matrix Multiplication

1. Read positive values m , n , p , and q .
2. Declare:
 - $A[m][n]$
 - $B[n][p]$
 - $C[p][q]$
3. Read all three matrices.
4. Calculate $AB = A \times B$, then $(AB) \times C$.
5. Calculate $BC = B \times C$, then $A \times (BC)$.
6. Compare the two final $m \times q$ matrices.
7. Display both results and state whether the associative property is verified.

Test Case Table

Test	Operation	Dimensions	Example	Expected Result
1	Addition	2×2	Three valid matrices	$(A+B)+C = A+(B+C)$
2	Addition	1×1	A=2, B=3, C=4	Both results = 9
3	Multiplication	A=2×2, B=2×2, C=2×2	Valid matrices	$(A \times B) \times C = A \times (B \times C)$
4	Multiplication	A=1×2, B=2×3, C=3×1	Valid matrices	Both final matrices are equal
5	Dimensions	m=0	Invalid	Ask for dimensions again
6	Menu	Choice 5	Invalid choice	Display error message
7	Exit	Choice 3	—	Program terminates

C Program

```
#include <stdio.h>

/* Function prototypes */
void readMatrix(int r, int c, int a[r][c]);
void printMatrix(int r, int c, int a[r][c]);

void addMatrix(int r, int c,
               int a[r][c],
               int b[r][c],
               int result[r][c]);

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
                   int b[c1][c2],
                   int result[r1][c2]);

int compareMatrix(int r, int c,
                  int a[r][c],
                  int b[r][c]);

void additionAssociative(void);
void multiplicationAssociative(void);

/* Read matrix elements */
void readMatrix(int r, int c, int a[r][c])
{
```

```

    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

/* Display a matrix */
void printMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%6d", a[i][j]);
        }
        printf("\n");
    }
}

/* Add two matrices */
void addMatrix(int r, int c,
               int a[r][c],
               int b[r][c],
               int result[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}

```

```

/* Multiply two matrices */
void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
                   int b[c1][c2],
                   int result[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (k = 0; k < c1; k++)
            {
                result[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

```

```

/* Compare two matrices */
int compareMatrix(int r, int c,
                  int a[r][c],
                  int b[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
            {
                return 0;
            }
        }
    }

    return 1;
}

```

```

/* Verify associative property of matrix addition */
void additionAssociative(void)
{
    int m, n;

    printf("\n--- Associative Property of Matrix Addition ---
\n");

    do
    {
        printf("Enter number of rows and columns (m n): ");
        scanf("%d %d", &m, &n);

        if (m <= 0 || n <= 0)
        {
            printf("Dimensions must be positive. Try again.
\n");
        }

        } while (m <= 0 || n <= 0);

    /* Variable-length arrays */
    int A[m][n];
    int B[m][n];
    int C[m][n];

    int AB[m][n];
    int BC[m][n];

    int left[m][n];
    int right[m][n];

    printf("\nEnter elements of matrix A:\n");
    readMatrix(m, n, A);

    printf("\nEnter elements of matrix B:\n");
    readMatrix(m, n, B);

    printf("\nEnter elements of matrix C:\n");
    readMatrix(m, n, C);

    /* Left side: (A + B) + C */
    addMatrix(m, n, A, B, AB);
    addMatrix(m, n, AB, C, left);

```

```

/* Right side: A + (B + C) */
addMatrix(m, n, B, C, BC);
addMatrix(m, n, A, BC, right);

printf("\n(A + B) + C:\n");
printMatrix(m, n, left);

printf("\nA + (B + C):\n");
printMatrix(m, n, right);

if (compareMatrix(m, n, left, right))
{
    printf("\nBoth matrices are equal.\n");
    printf("Hence, matrix addition is associative.\n");
    printf("(A + B) + C = A + (B + C)\n");
}
else
{
    printf("\nBoth matrices are not equal.\n");
    printf("Associative property is not verified.\n");
}
}

```

```

/* Verify associative property of matrix multiplication */
void multiplicationAssociative(void)
{
    int m, n, p, q;

    printf("\n--- Associative Property of Matrix
Multiplication ---\n");

    do
    {
        printf("Enter m, n, p and q: ");
        scanf("%d %d %d %d", &m, &n, &p, &q);

        if (m <= 0 || n <= 0 || p <= 0 || q <= 0)
        {
            printf("All dimensions must be positive. Try
again.\n");
        }

        } while (m <= 0 || n <= 0 || p <= 0 || q <= 0);
}

```

```

/*
    A = m x n
    B = n x p
    C = p x q
*/

int A[m][n];
int B[n][p];
int C[p][q];

/* A x B gives m x p */
int AB[m][p];

/* B x C gives n x q */
int BC[n][q];

/* Both final results are m x q */
int left[m][q];
int right[m][q];

printf("\nEnter elements of matrix A (%d x %d):\n", m,
n);
readMatrix(m, n, A);

printf("\nEnter elements of matrix B (%d x %d):\n", n,
p);
readMatrix(n, p, B);

printf("\nEnter elements of matrix C (%d x %d):\n", p,
q);
readMatrix(p, q, C);

/*
    Left side:
    (A x B) x C
*/
multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, p, q, AB, C, left);

/*
    Right side:
    A x (B x C)
*/
multiplyMatrix(n, p, q, B, C, BC);

```

```

multiplyMatrix(m, n, q, A, BC, right);

printf("\n(A x B) x C:\n");
printMatrix(m, q, left);

printf("\nA x (B x C):\n");
printMatrix(m, q, right);

if (compareMatrix(m, q, left, right))
{
    printf("\nBoth matrices are equal.\n");
    printf("Hence, matrix multiplication is associative.\n");
    printf("(A x B) x C = A x (B x C)\n");
}
else
{
    printf("\nBoth matrices are not equal.\n");
    printf("Associative property is not verified.\n");
}
}

```

```

/* Main function */
int main(void)
{
    int choice;

    do
    {

printf("\n===== \n");
        printf("                MATRIX OPERATIONS\n");

printf("===== \n");
        printf("1. Verify associative property of\naddition\n");
        printf("2. Verify associative property of\nmultiplication\n");
        printf("3. Exit\n");

printf("===== \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
}

```

```

switch (choice)
{
    case 1:
        additionAssociative();
        break;

    case 2:
        multiplicationAssociative();
        break;

    case 3:
        printf("\nExiting the program.\n");
        break;

    default:
        printf("\nInvalid choice. Please enter 1, 2,
or 3.\n");
}

} while (choice != 3);

return 0;
}

```

Execution - addition

```
[bhavya@Bhavyas-MacBook-Air ~ % clang day33.c -o day33
[bhavya@Bhavyas-MacBook-Air ~ % ./day33
```

```
=====
                        MATRIX OPERATIONS
=====
1. Verify associative property of addition
2. Verify associative property of multiplication
3. Exit
=====
Enter your choice: 1

--- Associative Property of Matrix Addition ---
Enter number of rows and columns (m n):  2 2

Enter elements of matrix A:
1 2
3 4

Enter elements of matrix B:
5 6
7 8

Enter elements of matrix C:
1 1
1 1

(A + B) + C:
  7   9
 11  13

A + (B + C):
  7   9
 11  13

Both matrices are equal.
Hence, matrix addition is associative.
(A + B) + C = A + (B + C)

=====
                        MATRIX OPERATIONS
=====
1. Verify associative property of addition
2. Verify associative property of multiplication
3. Exit
=====
```

Multiplication

```

Enter your choice: 2

--- Associative Property of Matrix Multiplication ---
Enter m, n, p and q: 2 2 2 2

Enter elements of matrix A (2 x 2):
1 2
3 4

Enter elements of matrix B (2 x 2):
5 6
7 8

Enter elements of matrix C (2 x 2):
1 0
0 1

(A x B) x C:
    19    22
    43    50

A x (B x C):
    19    22
    43    50

Both matrices are equal.
Hence, matrix multiplication is associative.
(A x B) x C = A x (B x C)

=====
                        MATRIX OPERATIONS
=====
1. Verify associative property of addition
2. Verify associative property of multiplication
3. Exit
=====
Enter your choice: █

```